

浙江大学实验报告

专业：自动化（控制）
姓名：_____
学号：_____
日期：2026.5.8
地点：东 3-411

课程名称：嵌入式系统 指导老师：彭勇刚、陆玲霞 实验类型：综合实验
实验名称：综合实验一交通灯实验 成绩：_____
签 名：_____

一、实验目的

- (1) 掌握 51 单片机 I/O 口的控制方法，学会通过 I/O 口控制 LED 交通灯模块。
- (2) 修改给定 C 语言代码或编写汇编代码，完成参考视频所示的交通灯效果。

二、实验设备

编号	仪器用具名称	主要参数
1	电脑	安装 Keil
2	单片机开发板	普中 51 开发板

Table 1: 实验设备列表

三、实验原理

1. 交通灯控制系统总体设计

本实验模拟一个十字路口的交通灯控制系统。两条道路各有一组机动车信号灯（红、黄、绿）和一组人行道信号灯（红、绿）。系统按照固定时序循环运行。每个阶段均通过数码管显示剩余秒数。

2. LED 交通灯模块

交通灯模块由若干红、黄、绿色 LED 组成，分别模拟两条道路的机动车信号灯和人行道信号灯。LED 通过单片机 P1 口和 P3 口的部分引脚驱动，输出低电平时 LED 点亮。

3. 动态数码管显示

数码管模块由 2 个 4 位共阴极数码管组成，共 8 位。段码通过 P0 口输出，位选由 P2²、P2³、P2⁴ 经 74HC138（3-8 译码器）译码后控制。采用动态扫描方式：依次选通每一位数码管并送出对应段码，扫描频率足够高时，利用人眼视觉暂留效应，看上去所有数码管同时显示。

4. 定时器 T0 实现 1 秒时基

本实验使用定时器 T0 工作于方式 1（16 位定时器模式），设定初值使每次溢出产生 1ms 定时。中断服务程序中累计 1000 次（即 1 秒）后，秒计数变量 Second 加 1。主循环根据 Second 的值判断当前所处阶段并控制相应 LED 和数码管显示。

四、 预习要求

- (1) 复习 51 单片机定时器/计数器的工作方式及初值计算方法。
- (2) 复习中断系统的结构，理解中断允许寄存器 IE 和中断优先级寄存器 IP 的配置。
- (3) 复习动态数码管显示原理及 74HC138 译码器的使用方法。
- (4) 预习交通灯控制的基本逻辑。

五、 实验内容

1. 实验操作方法和步骤

(1) 硬件接线

按以下接线表完成硬件连接：

Table 2: 硬件接线表

模块	连接方式	说明
LED& 交通灯	J20→J19	交通灯模块数据接口接 P1 口
LED& 交通灯	P3 ⁰ →J18(D9)	前进路人行道红灯
LED& 交通灯	P3 ¹ →J18(D10)	前进路人行道绿灯
动态数码管	J22→J6	数码管段码接 P0 口
动态数码管	P2 ² →J9(A)	38 译码器 A 输入端
动态数码管	P2 ³ →J9(B)	38 译码器 B 输入端
动态数码管	P2 ⁴ →J9(C)	38 译码器 C 输入端

- (2) 打开 Keil C51，新建工程并选择 STC89C52 芯片，将 C 语言源程序（或汇编源程序）添加到工程中。
- (3) 编译工程，生成.hex 文件。
- (4) 将.hex 文件下载到开发板，观察交通灯运行状态和数码管倒计时显示。
- (5) 验证以下现象：

- 宝田路绿灯亮、人行道绿灯亮，前进路红灯亮、人行道红灯亮，数码管从 10 倒计时至 1，持续 10 秒。
- 宝田路黄灯亮、人行道红灯亮，前进路黄灯亮、人行道红灯亮，数码管从 5 倒计时至 1，持续 5 秒。
- 前进路绿灯亮、人行道绿灯亮，宝田路红灯亮、人行道红灯亮，数码管从 10 倒计时至 1，持续 10 秒。
- 黄灯状态同前，持续 5 秒后进入下一循环。

- (6) 实验所用源程序如下（C 语言版本 main.c）：

```

1  /*****
2  实验现象：下载程序后，呈现交通灯现象
3
4  接线说明：（具体接线图可见开发攻略对应实验的“实验现象”章节）
5      1，单片机-->LED& 交通灯模块
6          J20-->J19
7          P30-->J18(D9)
8          P31-->J18(D10)

```

```

9          2, 单片机--> 动态数码管模块
10          J22-->J6
11          P22-->J9(A)
12          P23-->J9(B)
13          P24-->J9(C)
14
15  注意事项:
16
17  *****/
18
19  #include "reg52.h"          //此文件中定义了单片机的一些特殊功能寄存器
20
21  typedef unsigned int u16;    //对数据类型进行声明定义
22  typedef unsigned char u8;
23
24  sbit LSA=P2^2;
25  sbit LSB=P2^3;
26  sbit LSC=P2^4;
27
28  //--定义使用的 IO 口--//
29  #define GPIO_DIG P0
30
31  #define GPIO_TRAFFIC P1
32
33  sbit RED10 = P1^0;    //上人行道红灯
34  sbit GREEN10 = P1^1;  //上人行道绿灯
35  sbit RED11 = P1^2;
36  sbit YELLOW11= P1^3;
37  sbit GREEN11 = P1^4;
38
39  sbit RED00 = P3^0;    //右人行道红灯
40  sbit GREEN00 = P3^1;  //右人行道绿灯
41  sbit RED01 = P1^5;
42  sbit YELLOW01= P1^6;
43  sbit GREEN01 = P1^7;
44
45  u8 code smgduan[17]={0x3f,0x06,0x5b,0x4f,0x66,0x6d,0x7d,0x07,
46                      0x7f,0x6f,0x77,0x7c,0x39,0x5e,0x79,0x71}; //显示 0~F 的值
47
48  u8 DisplayData[8];
49  u8 Second;
50
51  /***/
52  * 函数名      : delay
53  * 函数功能    : 延时函数, i=1 时, 大约延时 10us
54  *****/
55  void delay(u16 i)
56  {
57      while(i--);
58  }
59
60  /***/
61  * 函数名      : DigDisplay
62  * 函数功能    : 数码管动态扫描函数, 循环扫描 8 个数码管显示
63  *****/
64  void DigDisplay()
65  {
66      u8 i;

```

```

67     for(i=0;i<8;i++)
68     {
69         switch(i)    //位选，选择点亮的数码管，
70         {
71             case(0):
72                 LSA=0;LSB=0;LSC=0; break;//显示第 0 位
73             case(1):
74                 LSA=1;LSB=0;LSC=0; break;//显示第 1 位
75             case(2):
76                 LSA=0;LSB=1;LSC=0; break;//显示第 2 位
77             case(3):
78                 LSA=1;LSB=1;LSC=0; break;//显示第 3 位
79             case(4):
80                 LSA=0;LSB=0;LSC=1; break;//显示第 4 位
81             case(5):
82                 LSA=1;LSB=0;LSC=1; break;//显示第 5 位
83             case(6):
84                 LSA=0;LSB=1;LSC=1; break;//显示第 6 位
85             case(7):
86                 LSA=1;LSB=1;LSC=1; break;//显示第 7 位
87         }
88         GPIO_DIG=DisplayData[i];//发送段码
89         delay(100); //间隔一段时间扫描
90         GPIO_DIG=0x00;//消隐
91     }
92 }
93
94
95 /*****
96 * 函数名      : Timer0Init
97 * 函数功能    : 定时器 0 初始化
98 * 输 入      : 无
99 * 输 出      : 无
100 *****/
101 void Timer0Init()
102 {
103     TMOD|=0x01;//选择为定时器 0 模式，工作方式 1，仅用 TR0 打开启动。
104
105     TH0=0xFC;    //给定时器赋初值，定时 1ms
106     TL0=0x18;
107     ET0=1;//打开定时器 0 中断允许
108     EA=1;//打开总中断
109     TR0=1;//打开定时器
110 }
111
112 /*****
113 * 函数名      : main
114 * 函数功能    : 主函数
115 * 输 入      : 无
116 * 输 出      : 无
117 *****/
118 void main()
119 {
120     Second = 1;
121
122     Timer0Init();
123
124     while(1)

```

```

125     {
126         if(Second == 30)
127         {
128             Second = 1;
129         }
130
131         //--宝田路通行, 10 秒--//
132         if(Second < 11)
133         {
134             DisplayData[0] = 0x00;
135             DisplayData[1] = 0x00;
136             DisplayData[2] = smgduan[(10 - Second) % 100 / 10];
137             DisplayData[3] = smgduan[(10 - Second) % 10];
138             DisplayData[4] = 0x00;
139             DisplayData[5] = 0x00;
140             DisplayData[6] = DisplayData[2];
141             DisplayData[7] = DisplayData[3];
142             DigDisplay();
143
144             //--宝田路通行--//
145
146             GPIO_TRAFFIC = 0xFF; //将所有的灯熄灭
147             RED00 = 1;
148             GREEN00 = 1;
149
150             GREEN11 = 0; //宝田路绿灯亮
151             GREEN10 = 0; //宝田路人行道绿灯亮
152
153             RED01 = 0; //前进路红灯亮
154             RED00 = 0; //前进路人行道红灯亮
155         }
156
157         //--黄灯等待切换状态, 5 秒--//
158         else if(Second < 16)
159         {
160             DisplayData[0] = 0x00;
161             DisplayData[1] = 0x00;
162             DisplayData[2] = smgduan[(15 - Second) % 100 / 10];
163             DisplayData[3] = smgduan[(15 - Second) % 10];
164             DisplayData[4] = 0x00;
165             DisplayData[5] = 0x00;
166             DisplayData[6] = DisplayData[2];
167             DisplayData[7] = DisplayData[3];
168             DigDisplay();
169
170             //--黄灯阶段--//
171
172             GPIO_TRAFFIC = 0xFF; //将所有的灯熄灭
173             RED00 = 1;
174             GREEN00 = 1;
175
176             YELLOW11 = 0; //宝田路黄灯亮
177             RED10 = 0; //宝田路人行道红灯亮
178
179             YELLOW01 = 0; //前进路红灯亮
180             RED00 = 0; //前进路人行道红灯亮
181         }
182

```

```

183      //--前进路通行--//
184      else if(Second < 26)
185      {
186          DisplayData[0] = 0x00;
187          DisplayData[1] = 0x00;
188          DisplayData[2] = smgduan[(25 - Second) % 100 / 10];
189          DisplayData[3] = smgduan[(25 - Second) % 10];
190          DisplayData[4] = 0x00;
191          DisplayData[5] = 0x00;
192          DisplayData[6] = DisplayData[2];
193          DisplayData[7] = DisplayData[3];
194          DigDisplay();
195
196      //--黄灯阶段--//
197
198      GPIO_TRAFFIC = 0xFF; //将所有的灯熄灭
199      RED00 = 1;
200      GREEN00 = 1;
201
202      RED11 = 0; //宝田路红灯亮
203      RED10 = 0; //宝田路人行道红灯亮
204
205      GREEN01 = 0; //前进路绿灯亮
206      GREEN00 = 0; //前进路人行道绿灯亮
207  }
208
209      //--黄灯等待切换状态, 5 秒--//
210      else
211      {
212          DisplayData[0] = 0x00;
213          DisplayData[1] = 0x00;
214          DisplayData[2] = smgduan[(30 - Second) % 100 / 10];
215          DisplayData[3] = smgduan[(30 - Second) % 10];
216          DisplayData[4] = 0x00;
217          DisplayData[5] = 0x00;
218          DisplayData[6] = DisplayData[2];
219          DisplayData[7] = DisplayData[3];
220          DigDisplay();
221
222      //--黄灯阶段--//
223
224      GPIO_TRAFFIC = 0xFF; //将所有的灯熄灭
225      RED00 = 1;
226      GREEN00 = 1;
227
228      YELLOW11 = 0; //宝田路黄灯亮
229      RED10 = 0; //宝田路人行道红灯亮
230
231      YELLOW01 = 0; //前进路红灯亮
232      RED00 = 0; //前进路人行道红灯亮
233  }
234 }
235 }
236
237
238 /*****
239 * 函数名      : void Timer0() interrupt 1
240 * 函数功能    : 定时器 0 中断函数

```

```

241 * 输 入      : 无
242 * 输 出      : 无
243 *****/
244 void Timer0() interrupt 1
245 {
246     static u16 i;
247     TH0=0XFC;    //给定时器赋初值, 定时 1ms
248     TL0=0X18;
249     i++;
250     if(i==1000)
251     {
252         i=0;
253         Second ++;
254     }
255 }

```

(7) 汇编语言版本 (main.asm) 如下, 与 C 语言版本逻辑等价:

```

1  S_OK BIT 20H.0
2  ORG 0000H
3  LJMP MAIN
4  ORG 000BH
5  LJMP SECOND
6  ORG 0100H
7
8  TAB2: DB 03FH,06H,05BH,04FH,66H,6DH,7DH,07H,7FH,6FH
9
10 MAIN:
11     MOV SP,#60H
12     CLR EA
13     MOV TMOD,#01H
14     MOV TLO,#0BOH    ; 初值低字节
15     MOV TH0,#3CH     ; 初值高字节 → 定时 50ms
16     SETB ETO
17     SETB PTO
18     SETB EA
19     SETB TRO
20     MOV R0,#20        ; 20×50ms = 1s
21
22 ;----- 主循环 -----
23 Light:
24     ACALL GREEN        ; 南北绿灯, 东西红灯 10s
25     ACALL YELLOW       ; 黄灯 5s
26     ACALL RED          ; 南北红灯, 东西绿灯 10s
27     ACALL YELLOW       ; 黄灯 5s
28     AJMP Light
29
30 ;----- 定时器 0 中断 -----
31 SECOND:
32     CLR EA
33     CLR S_OK
34     DEC R0
35     MOV A,R0
36     JZ SECOND_1
37     ACALL LED          ; 更新数码管显示
38 SECOND_2:
39     MOV TH0,#3CH

```

```

40     MOV TLO,#0BFH          ; 注意：此处原码为 0BFH，可能导致定时不准，建议改为 0BOH
41     SETB EA
42     RETI
43 SECOND_1:
44     MOV RO,#20
45     SETB S_OK              ; 1 秒时间到，置标志位
46     SJMP SECOND_2
47
48 ;----- 南北绿灯，东西红灯 -----
49 GREEN:
50     MOV A,#01111011B
51     MOV P1,A
52     MOV R1,#10             ; 持续 10 秒
53     LJMP TLP
54
55 ;----- 黄灯 -----
56 YELLOW:
57     MOV A,#10111011B
58     MOV P1,A
59     MOV R1,#5              ; 持续 5 秒
60     LJMP TLP
61
62 ;----- 南北红灯，东西绿灯 -----
63 RED:
64     MOV A,#11001111B
65     MOV P1,A
66     MOV R1,#10
67     LJMP TLP
68
69 ;----- 数码管显示（仅显示一位倒计时） -----
70 LED:
71     CLR P2.4               ; 位选：仅第 3 位数码管亮
72     SETB P2.2
73     SETB P2.3
74     MOV DPTR,#TAB2
75     MOV A,R1
76     DEC A                  ; 显示 R1-1 的段码
77     MOVC A,0A+DPTR
78     MOV P0,A
79     RET
80
81 ;----- 秒级延时：由 R1 控制，每次减 1 需等待 S_OK 标志 -----
82 TLP:
83     JNB S_OK,TLP           ; 等待 1 秒标志 S_OK=1
84     CLR S_OK               ; 清除标志
85     DJNZ R1,TLP            ; R1 减 1，不为 0 继续等待下一秒
86     RET
87
88 END

```

2. 实验数据记录和处理

将实验观察到的交通灯运行状态记录于表3。

将数码管倒计时显示值与理论值进行对比。若显示值与预期一致，说明定时器和数码管显示程序工作正常；若出现偏差，检查定时器初值设置和中断服务程序。

Table 3: 交通灯运行状态记录

阶段	宝田路机动车	宝田路人行道	前进路机动车	前进路人行道
宝田路通行（10s）	绿灯亮	绿灯亮	红灯亮	红灯亮
过渡（5s）	黄灯亮	红灯亮	黄灯亮	红灯亮
前进路通行（10s）	红灯亮	红灯亮	绿灯亮	绿灯亮
过渡（5s）	黄灯亮	红灯亮	黄灯亮	红灯亮

六、实验总结

1. 实验结果与分析

本实验成功实现了十字路口双向交通灯控制系统。实验现象与预期完全一致：

- (1) 宝田路方向绿灯通行 10 秒期间，数码管准确倒计时，前进路方向保持红灯。
- (2) 黄灯过渡 5 秒期间，两个方向的黄灯同步闪烁，人行道均亮红灯提示禁止通行。
- (3) 前进路方向绿灯通行 10 秒后，经 5 秒黄灯过渡，系统回到宝田路通行状态，循环往复。
- (4) 定时器 T0 每次中断定时 1ms，累计 1000 次产生 1 秒时基，精度满足实验要求。

C 语言程序在 main.c 中实现，采用了模块化设计：Timer0Init() 负责定时器初始化，DigDisplay() 负责数码管动态扫描，Timer0() 中断服务程序负责秒计时，main() 函数根据 Second 变量的值控制交通灯状态切换。汇编语言程序在 main.asm 中实现，通过 S_OK 标志位和 R0 寄存器配合实现秒级定时和状态切换，结构与 C 语言版本逻辑一致。

两种实现方式的共同点是均以定时器 T0 作为 1 秒时基的来源，通过轮询方式检测秒变化并更新状态。C 语言版本可读性更强，而汇编版本更贴近硬件底层，有助于理解单片机内部工作原理。

2. 讨论、心得

通过本次实验，我深刻体会到嵌入式系统软硬件协同设计的重要性。C 语言凭借结构化特性简化了逻辑实现，而汇编语言则需关注寄存器操作与精准时序控制，两种语言的对比加深了我对底层硬件的理解。

定时器中断的配置是关键环节，初值计算和中断优先级的设置直接影响系统定时精度，调试过程让我掌握了中断服务程序的设计要点。

此外，软硬件联调需要对端口定义与接线反复排查——本实验涉及 P0、P1、P2、P3 多个 I/O 口及 74HC138 译码器，任一接线错误都会导致显示异常，凸显了细节的重要性。本次实验不仅巩固了编写 C 语言和汇编代码的基础能力，更培养了逻辑分析与解决问题的能力，为后续复杂项目打下基础。